

Lab 4 – VibeCoding Chat History

Assignment: 60_number_guess

Lab: Lab 4 – VibeCoding

Topic: Python Pygame Number Guessing Game

1. Initial Request — Empty-Input Crash

User

I am working on a Python Pygame number guessing game.

I need you to fix only the empty-input crash first.

When the player presses Enter or clicks SUBMIT without entering any digits, the current code crashes because it attempts to convert an empty string to int.

Please:

1. Inspect the code I provide.
2. Identify the exact cause of the crash.
3. Modify the code so empty input does not call int().
4. Preserve the attempt count when the input is empty.
5. Set an appropriate feedback message asking the player to enter a valid number.
6. Do not implement the other features yet.
7. Give me the complete corrected file, not just a patch.

Here is my current game_engine.py:

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
```

```

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)

def submit_guess(self):
    if self.game_won:
        return

    # BUG SYMPTOM:
    # Submitting an empty input box crashes the game immediately.
    guess = int(self.input_box.text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
    elif guess > self.secret_number:
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)
    else:
        self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
        self.feedback_color = (80, 220, 90)
        self.game_won = True

def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0
    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

```

```

if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_RETURN:
        self.submit_guess()
    elif event.key == pygame.K_r and self.game_won:
        self.reset()

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
    if self.submit_btn.collidepoint(event.pos):
        self.submit_guess()

def update(self):
    pass

def render(self, screen):
    screen.fill((30, 34, 42))

    title_surf = self.font_title.render("Number Guessing Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 35))

    attempts_surf = self.font_medium.render(f"Attempts: {self.attempts}", True, (180, 185,
195))
    screen.blit(attempts_surf, (self.width // 2 - attempts_surf.get_width() // 2, 95))
    self.input_box.render(screen)

    pygame.draw.rect(screen, (50, 150, 80), self.submit_btn, border_radius=6)
    pygame.draw.rect(screen, (220, 220, 220), self.submit_btn, width=2, border_radius=6)
    btn_text = self.font_btn.render("SUBMIT", True, (255, 255, 255))
    screen.blit(
        btn_text,
        (self.submit_btn.centerx - btn_text.get_width() // 2, self.submit_btn.centery -
btn_text.get_height() // 2),
    )

    feedback_surf = self.font_medium.render(self.feedback_msg, True, self.feedback_color)
    screen.blit(feedback_surf, (self.width // 2 - feedback_surf.get_width() // 2, 235))

    if self.game_won:
        restart_surf = self.font_medium.render("Press [R] to Start a New Game", True, (255,
220, 80))
        screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, 295))

```

2. Task 1 — Fix Empty-Input Crash

AI Response

The crash happens at:

```
guess = int(self.input_box.text)
```

When the input box is empty, `self.input_box.text` is "", and `int("")` raises a `ValueError`.

The fix is to check whether the input is empty before calling `int()`. In that case, we return immediately, so the attempt count is unchanged and the game does not crash.

Here is the complete corrected `game_engine.py` with only this change:

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)
        self.game_won = False

        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)

    def submit_guess(self):
        if self.game_won:
            return

        # Check for empty input before converting to int.
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
```

```

        self.feedback_color = (240, 200, 80)
        return

    guess = int(self.input_box.text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
    elif guess > self.secret_number:
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)
    else:
        self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
        self.feedback_color = (80, 220, 90)
        self.game_won = True

    def reset(self):
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)
        self.game_won = False
        self.input_box.clear()

    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:
                self.submit_guess()
            elif event.key == pygame.K_r and self.game_won:
                self.reset()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

    def update(self):
        pass

    def render(self, screen):

```

```

screen.fill((30, 34, 42))

title_surf = self.font_title.render(
    "Number Guessing Arena", True, (245, 245, 245)
)
screen.blit(
    title_surf,
    (
        self.width // 2 - title_surf.get_width() // 2,
        35,
    ),
)

attempts_surf = self.font_medium.render(
    f'Attempts: {self.attempts}', True, (180, 185, 195)
)
screen.blit(
    attempts_surf,
    (
        self.width // 2 - attempts_surf.get_width() // 2,
        95,
    ),
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (50, 150, 80),
    self.submit_btn,
    border_radius=6,
)
pygame.draw.rect(
    screen,
    (220, 220, 220),
    self.submit_btn,
    width=2,
    border_radius=6,
)

btn_text = self.font_btn.render(
    "SUBMIT", True, (255, 255, 255)
)
screen.blit(

```

```

        btn_text,
        (
            self.submit_btn.centerx - btn_text.get_width() // 2,
            self.submit_btn.centery - btn_text.get_height() // 2,
        ),
    )

    feedback_surf = self.font_medium.render(
        self.feedback_msg, True, self.feedback_color
    )
    screen.blit(
        feedback_surf,
        (
            self.width // 2 - feedback_surf.get_width() // 2,
            235,
        ),
    )

    if self.game_won:
        restart_surf = self.font_medium.render(
            "Press [R] to Start a New Game", True, (255, 220, 80)
        )
        screen.blit(
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                295,
            ),
        )

```

Changes made

Only this block was added before `int()`:

```

if not self.input_box.text.strip():
    self.feedback_msg = "Please enter a valid number."
    self.feedback_color = (240, 200, 80)
    return

```

So now:

- Empty input + Enter → no crash
- Empty input + SUBMIT → no crash
- Attempts remain unchanged

- Feedback says "Please enter a valid number."
- Non-empty input continues through the existing guessing logic
- No other features were added.

3. Task 2 — Dynamic Possible Range

User

Now implement Task 2 from the README.

Add dynamic range hints to the number guessing game.

Requirements:

1. Maintain `low_bound`, initially 1.
2. Maintain `high_bound`, initially 100.
3. After every valid guess, update the appropriate bound based on whether the guess is too low or too high.
4. Display the current possible range on the game screen, for example:
"Current Possible Range: 24 - 68"
5. The range should help the player narrow down the possible secret number.
6. Empty input should still be handled safely.
7. Do not break the existing game functionality.
8. Give me the complete modified file(s), not just a patch.

Use the existing code structure and make the smallest reasonable changes.

AI Response

Absolutely. For Task 2, we only need to extend the existing `GameEngine` with `low_bound` and `high_bound`, update them after valid guesses, and display the range.

The important part is that empty input returns before the attempt count or bounds are changed.

Here is the complete modified `game_engine.py`:

```
import random
import pygame
from game.text_box import TextBox
```

```
class GameEngine:
```

```

def __init__(self, width, height):
    self.width = width
    self.height = height
    self.secret_number = random.randint(1, 100)

    self.low_bound = 1
    self.high_bound = 100

    self.attempts = 0
    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False

    self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
    self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

    self.font_title = pygame.font.SysFont(None, 42)
    self.font_medium = pygame.font.SysFont(None, 28)
    self.font_btn = pygame.font.SysFont(None, 26)

def submit_guess(self):
    if self.game_won:
        return

    # Check for empty input before converting to int.
    if not self.input_box.text.strip():
        self.feedback_msg = "Please enter a valid number."
        self.feedback_color = (240, 200, 80)
        return

    guess = int(self.input_box.text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.low_bound = guess + 1
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)

    elif guess > self.secret_number:
        self.high_bound = guess - 1
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)

```

```

else:
    self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
    self.feedback_color = (80, 220, 90)
    self.game_won = True

def reset(self):
    self.secret_number = random.randint(1, 100)

    self.low_bound = 1
    self.high_bound = 100

    self.attempts = 0
    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            self.submit_guess()
        elif event.key == pygame.K_r and self.game_won:
            self.reset()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_guess()

def update(self):
    pass

def render(self, screen):
    screen.fill((30, 34, 42))

    title_surf = self.font_title.render(
        "Number Guessing Arena", True, (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,

```

```

        35,
    ),
)

attempts_surf = self.font_medium.render(
    f"Attempts: {self.attempts}", True, (180, 185, 195)
)
screen.blit(
    attempts_surf,
    (
        self.width // 2 - attempts_surf.get_width() // 2,
        95,
    ),
)

range_surf = self.font_medium.render(
    f"Current Possible Range: {self.low_bound} - {self.high_bound}",
    True,
    (200, 210, 220),
)
screen.blit(
    range_surf,
    (
        self.width // 2 - range_surf.get_width() // 2,
        125,
    ),
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (50, 150, 80),
    self.submit_btn,
    border_radius=6,
)
pygame.draw.rect(
    screen,
    (220, 220, 220),
    self.submit_btn,
    width=2,
    border_radius=6,
)

```

```

btn_text = self.font_btn.render(
    "SUBMIT", True, (255, 255, 255)
)
screen.blit(
    btn_text,
    (
        self.submit_btn.centerx - btn_text.get_width() // 2,
        self.submit_btn.centery - btn_text.get_height() // 2,
    ),
)

feedback_surf = self.font_medium.render(
    self.feedback_msg, True, self.feedback_color
)
screen.blit(
    feedback_surf,
    (
        self.width // 2 - feedback_surf.get_width() // 2,
        235,
    ),
)

if self.game_won:
    restart_surf = self.font_medium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            295,
        ),
    )

```

Task 2 changes

The changes are intentionally small:

- Added:

```
self.low_bound = 1
```

```
self.high_bound = 100
```

- If the guess is too low:

```
self.low_bound = guess + 1
```

- If the guess is too high:

```
self.high_bound = guess - 1
```

- Added the displayed range:

```
Current Possible Range: 24 - 68
```

- reset() restores the range to 1 - 100.
- Empty input is checked before int(), so it does not increment attempts or change the range.

4. Task 3 — Last 5 Guess History

User

Now implement Task 3 from the README.

Add a visual guess history panel to the Pygame interface.

Requirements:

1. Store the player's previous valid guesses.
2. Display the last 5 guesses only.
3. For each guess, clearly indicate whether it was TOO HIGH or TOO LOW.
4. Use simple visual labels, arrows, or text such as:
42 ↓ TOO LOW
78 ↑ TOO HIGH
5. Do not add empty submissions to the history.
6. Do not break the dynamic range feature or the existing game.
7. Keep the interface readable and avoid overlapping existing UI elements.
8. Give me the complete modified code needed for this feature.

Please work with the existing code rather than rewriting the entire project unnecessarily.

AI Response

Yes. For Task 3, we can make small changes only to `game_engine.py`. The history will store valid guesses, keep only the latest 5, and show ↓ TOO LOW / ↑ TOO HIGH.

I'll also keep the dynamic range and empty-input handling from Tasks 1 and 2.

Here is the complete modified `game_engine.py`:

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)

        self.low_bound = 1
        self.high_bound = 100

        self.attempts = 0
        self.guess_history = []

        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)
        self.game_won = False

        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)
        self.font_history = pygame.font.SysFont(None, 24)

    def submit_guess(self):
        if self.game_won:
            return

        # Check for empty input before converting to int.
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
            self.feedback_color = (240, 200, 80)
```

```

        return

    guess = int(self.input_box.text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.low_bound = guess + 1

        # Store valid guess in history.
        self.guess_history.append((guess, "TOO LOW"))

        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)

    elif guess > self.secret_number:
        self.high_bound = guess - 1

        # Store valid guess in history.
        self.guess_history.append((guess, "TOO HIGH"))

        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)

    else:
        # Store the correct guess in history as well.
        self.guess_history.append((guess, "CORRECT"))

        self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
        self.feedback_color = (80, 220, 90)
        self.game_won = True

    # Keep only the last 5 guesses.
    self.guess_history = self.guess_history[-5:]

def reset(self):
    self.secret_number = random.randint(1, 100)

    self.low_bound = 1
    self.high_bound = 100

    self.attempts = 0
    self.guess_history = []

```

```

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            self.submit_guess()
        elif event.key == pygame.K_r and self.game_won:
            self.reset()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_guess()

def update(self):
    pass

def render(self, screen):
    screen.fill((30, 34, 42))

    title_surf = self.font_title.render(
        "Number Guessing Arena", True, (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            35,
        ),
    )

    attempts_surf = self.font_medium.render(
        f"Attempts: {self.attempts}", True, (180, 185, 195)
    )
    screen.blit(
        attempts_surf,
        (
            self.width // 2 - attempts_surf.get_width() // 2,
            95,

```

```

    ),
)

range_surf = self.font_medium.render(
    f"Current Possible Range: {self.low_bound} - {self.high_bound}",
    True,
    (200, 210, 220),
)
screen.blit(
    range_surf,
    (
        self.width // 2 - range_surf.get_width() // 2,
        125,
    ),
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (50, 150, 80),
    self.submit_btn,
    border_radius=6,
)
pygame.draw.rect(
    screen,
    (220, 220, 220),
    self.submit_btn,
    width=2,
    border_radius=6,
)

btn_text = self.font_btn.render(
    "SUBMIT", True, (255, 255, 255)
)
screen.blit(
    btn_text,
    (
        self.submit_btn.centerx - btn_text.get_width() // 2,
        self.submit_btn.centery - btn_text.get_height() // 2,
    ),
)

feedback_surf = self.font_medium.render(

```

```

        self.feedback_msg, True, self.feedback_color
    )
    screen.blit(
        feedback_surf,
        (
            self.width // 2 - feedback_surf.get_width() // 2,
            235,
        ),
    )
)

```

```

# Guess history panel
history_title = self.font_medium.render(
    "Guess History", True, (245, 245, 245)
)
screen.blit(
    history_title,
    (
        self.width // 2 - history_title.get_width() // 2,
        285,
    ),
)

```

```

history_y = 320

```

```

for guess, result in reversed(self.guess_history):
    if result == "TOO LOW":
        history_text = f"{guess} \u2193 TOO LOW"
        history_color = (80, 160, 240)
    elif result == "TOO HIGH":
        history_text = f"{guess} \u2191 TOO HIGH"
        history_color = (240, 100, 80)
    else:
        history_text = f"{guess} \u2713 CORRECT"
        history_color = (80, 220, 90)

    history_surf = self.font_history.render(
        history_text, True, history_color
    )
    screen.blit(
        history_surf,
        (
            self.width // 2 - history_surf.get_width() // 2,
            history_y,
        ),
    )

```

```

    )

    history_y += 28

if self.game_won:
    restart_surf = self.font_medium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            history_y + 15,
        ),
    )

```

Task 3 changes

1. History storage

```
self.guess_history = []
```

Each valid guess is stored as:

```
(42, "TOO LOW")
```

```
(78, "TOO HIGH")
```

The correct guess is also stored:

```
(65, "CORRECT")
```

2. Only the last 5 are retained

```
self.guess_history = self.guess_history[-5:]
```

3. Empty submissions aren't stored

The empty-input check happens before the history code:

```
if not self.input_box.text.strip():
```

So pressing Enter or clicking SUBMIT with nothing entered does not affect attempts, range, or history.

4. Visual history

It will display entries like:

Guess History

78 ↑ TOO HIGH

42 ↓ TOO LOW

65 ✓ CORRECT

The newest guess appears first.

5. Dynamic range is preserved

Current Possible Range: 43 - 77

continues updating exactly as in Task 2.

6. Reset clears the history

Pressing R after winning starts a completely fresh game with:

Attempts: 0

Current Possible Range: 1 - 100

Guess History

No other project files need to be changed for this task.

5. Task 4 — 7-Attempt Limit and GAME OVER

User

Now implement Task 4 from the README.

Add a maximum attempt limit and GAME OVER state.

Requirements:

1. Set the maximum number of attempts to 7.
2. A valid guess should increment the attempt counter.
3. Empty submissions must not consume an attempt.
4. If the player guesses the secret number, show the existing victory state.
5. If the player reaches 7 valid guesses without finding the number, show a GAME OVER state.
6. Reveal the secret number on the GAME OVER screen.
7. Tell the player to press R to restart.
8. Restarting with R must reset:
 - secret number
 - attempts
 - feedback
 - dynamic range
 - guess history
 - game-over state
9. Make sure the existing features continue working.
10. Give me the complete modified code required.

Please inspect the current implementation carefully before changing it and avoid unnecessary rewrites.

AI Response

Yes. We can build Task 4 directly on the current Task 3 implementation with minimal changes.

One important detail: the 7th valid guess is still processed normally. If it is correct, the existing victory state is shown. If it is wrong, `game_over` becomes `True` and the secret number is revealed.

Only `game_engine.py` needs to change.

Here is the complete modified `game_engine.py`:

```
import random
import pygame
from game.text_box import TextBox
```

```
class GameEngine:
    def __init__(self, width, height):
```

```

self.width = width
self.height = height
self.secret_number = random.randint(1, 100)

self.low_bound = 1
self.high_bound = 100

self.attempts = 0
self.max_attempts = 7
self.guess_history = []

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.game_over = False

self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

self.font_title = pygame.font.SysFont(None, 42)
self.font_medium = pygame.font.SysFont(None, 28)
self.font_btn = pygame.font.SysFont(None, 26)
self.font_history = pygame.font.SysFont(None, 24)

def submit_guess(self):
    if self.game_won or self.game_over:
        return

    # Empty input does not consume an attempt.
    if not self.input_box.text.strip():
        self.feedback_msg = "Please enter a valid number."
        self.feedback_color = (240, 200, 80)
        return

    guess = int(self.input_box.text)

    # Every valid guess consumes one attempt.
    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.low_bound = guess + 1

        self.guess_history.append((guess, "TOO LOW"))

```

```

        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)

elif guess > self.secret_number:
    self.high_bound = guess - 1

    self.guess_history.append((guess, "TOO HIGH"))

    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)

else:
    self.guess_history.append((guess, "CORRECT"))

    self.feedback_msg = (
        f"CORRECT! Found in {self.attempts} attempts."
    )
    self.feedback_color = (80, 220, 90)
    self.game_won = True

# Keep only the last 5 guesses.
self.guess_history = self.guess_history[-5:]

# If the player has used all 7 attempts without winning,
# end the game.
if self.attempts >= self.max_attempts and not self.game_won:
    self.game_over = True
    self.feedback_msg = (
        f"GAME OVER! The number was {self.secret_number}."
    )
    self.feedback_color = (240, 100, 80)

def reset(self):
    self.secret_number = random.randint(1, 100)

    self.low_bound = 1
    self.high_bound = 100

    self.attempts = 0
    self.guess_history = []

    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)

```

```

self.game_won = False
self.game_over = False

self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            self.submit_guess()

        elif event.key == pygame.K_r and (self.game_won or self.game_over):
            self.reset()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_guess()

def update(self):
    pass

def render(self, screen):
    screen.fill((30, 34, 42))

    title_surf = self.font_title.render(
        "Number Guessing Arena",
        True,
        (245, 245, 245),
    )
    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            35,
        ),
    )

    attempts_surf = self.font_medium.render(
        f"Attempts: {self.attempts}/{self.max_attempts}",
        True,
        (180, 185, 195),
    )
    screen.blit(

```

```

    attempts_surf,
    (
        self.width // 2 - attempts_surf.get_width() // 2,
        95,
    ),
)

range_surf = self.font_medium.render(
    f"Current Possible Range: {self.low_bound} - {self.high_bound}",
    True,
    (200, 210, 220),
)
screen.blit(
    range_surf,
    (
        self.width // 2 - range_surf.get_width() // 2,
        125,
    ),
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (50, 150, 80),
    self.submit_btn,
    border_radius=6,
)
pygame.draw.rect(
    screen,
    (220, 220, 220),
    self.submit_btn,
    width=2,
    border_radius=6,
)

btn_text = self.font_btn.render(
    "SUBMIT",
    True,
    (255, 255, 255),
)
screen.blit(
    btn_text,
    (

```

```

        self.submit_btn.centerx - btn_text.get_width() // 2,
        self.submit_btn.centery - btn_text.get_height() // 2,
    ),
)

feedback_surf = self.font_medium.render(
    self.feedback_msg,
    True,
    self.feedback_color,
)
screen.blit(
    feedback_surf,
    (
        self.width // 2 - feedback_surf.get_width() // 2,
        235,
    ),
)

# Guess history panel
history_title = self.font_medium.render(
    "Guess History",
    True,
    (245, 245, 245),
)
screen.blit(
    history_title,
    (
        self.width // 2 - history_title.get_width() // 2,
        285,
    ),
)

history_y = 320

for guess, result in reversed(self.guess_history):
    if result == "TOO LOW":
        history_text = f"{guess} \u2193 TOO LOW"
        history_color = (80, 160, 240)

    elif result == "TOO HIGH":
        history_text = f"{guess} \u2191 TOO HIGH"
        history_color = (240, 100, 80)

    else:

```

```

        history_text = f"{guess} \u2713 CORRECT"
        history_color = (80, 220, 90)

        history_surf = self.font_history.render(
            history_text,
            True,
            history_color,
        )
        screen.blit(
            history_surf,
            (
                self.width // 2 - history_surf.get_width() // 2,
                history_y,
            ),
        )

        history_y += 28

# Victory state
if self.game_won:
    restart_surf = self.font_medium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            history_y + 15,
        ),
    )

# Game over state
elif self.game_over:
    game_over_surf = self.font_medium.render(
        "GAME OVER",
        True,
        (240, 100, 80),
    )
    screen.blit(
        game_over_surf,
        (
            self.width // 2 - game_over_surf.get_width() // 2,

```

```

        history_y + 15,
    ),
)

secret_surf = self.font_medium.render(
    f"The secret number was {self.secret_number}",
    True,
    (245, 245, 245),
)
screen.blit(
    secret_surf,
    (
        self.width // 2 - secret_surf.get_width() // 2,
        history_y + 50,
    ),
)

restart_surf = self.font_medium.render(
    "Press [R] to Restart",
    True,
    (255, 220, 80),
)
screen.blit(
    restart_surf,
    (
        self.width // 2 - restart_surf.get_width() // 2,
        history_y + 85,
    ),
)

```

What changed for Task 4

Maximum attempts:

```
self.max_attempts = 7
```

Valid guesses increment attempts:

```
self.attempts += 1
```

This happens only after the empty-input check, so:

Empty input → 0 attempts used

Valid input → 1 attempt used

Game over after the 7th incorrect guess:

```
if self.attempts >= self.max_attempts and not self.game_won:  
    self.game_over = True
```

The secret number is then displayed.

The game stops accepting guesses after GAME OVER:

```
if self.game_won or self.game_over:  
    return
```

R restarts both victory and game-over states:

```
elif event.key == pygame.K_r and (self.game_won or self.game_over):  
    self.reset()
```

And `reset()` clears everything required:

- Secret number → new random number
- Attempts → 0
- Feedback → initial message
- Range → 1 - 100
- History → empty
- `game_won` → `False`
- `game_over` → `False`

The existing **Task 2 dynamic range** and **Task 3 five-guess history** remain intact.

6. Final Implementation

The final implementation from the conversation contains the following features:

Task 1 — Empty Input Handling

- Empty Enter/SUBMIT does not crash.
- `int()` is not called for empty input.
- Empty input does not consume an attempt.
- Feedback asks the player to enter a valid number.

Task 2 — Dynamic Range

- Initial range is 1 - 100.
- A low guess changes `low_bound`.
- A high guess changes `high_bound`.
- The current possible range is displayed.

Task 3 — Guess History

- Valid guesses are stored.
- Only the last 5 guesses are displayed.
- Guesses are labelled `TOO LOW`, `TOO HIGH`, or `CORRECT`.
- Empty submissions are not added.

Task 4 — Maximum Attempts

- Maximum attempts are set to 7.
 - Every valid guess increments the attempt counter.
 - Empty submissions do not consume attempts.
 - Correct guesses retain the victory state.
 - Seven incorrect valid guesses produce `GAME OVER`.
 - The secret number is revealed.
 - `R` restarts the game.
 - Restart resets the secret number, attempts, feedback, range, history, and game-over state.
-

Final `game_engine.py` State

The final Task 4 response above contains the complete implementation used as the final version in this conversation. For submission, **you can include the full Task 4 code from Section 5**, since it incorporates the earlier Tasks 1–3.